

Developer Guide — Project Overview

This guide explains the purpose of each app folder, main files to inspect for bugs, how apps connect, and quick pointers like "if you want to fix a bug in HR go here".

App: accounts

Path: C:\Users\Asadullah\OneDrive\Desktop\ERP\leap-erp\accounts

****Primary purpose:**** (No module docstring found; inferred from filenames below)

Files:

- __init__.py
- admin.py
- apps.py
- backends.py
- decorators.py
- forms.py
- models.py
- permissions.py
- tests.py
- urls.py
- validators.py
- views.py

Key places to look when troubleshooting:

- models.py — database schema and model methods (data validation/business rules tied to persistence)
- views.py — request handling, business workflows, API endpoints and templates rendered
- forms.py — input validation and form-level logic
- urls.py — routing (which endpoint maps to which view)
- admin.py — admin UI customisations (if the bug appears in Django admin)

models.py (first lines)

```
from django.conf import settings
from django.contrib.auth.models import AbstractUser
from django.db import models
```

```
class Role(models.Model):
    """User roles for access control"""
    SUPER_ADMIN = 'super_admin'
    ADMIN = 'admin'
    MANAGER = 'manager'
    SALES_REP = 'sales_rep'
    PROCUREMENT_MGR = 'procurement_mgr'
    PROCUREMENT_OFF = 'procurement_off'
    PROPOSAL_HEAD = 'proposal_head'
    PROPOSAL_REP = 'proposal_rep'
    FINANCE_HEAD = 'finance_head'
    FINANCE_MANAGER = 'finance_manager'
    FINANCE_REP = 'finance_rep'
    DEVELOPER = 'developer'
    AI_HEAD = 'ai_head'
```

views.py (first lines)

```
import json
from django.shortcuts import render, redirect, get_object_or_404
from django.contrib.auth import login, logout
from django.contrib.auth.views import LoginView, LogoutView
from django.contrib.auth.decorators import login_required
from django.contrib import messages
from django.views.generic import ListView, CreateView, UpdateView, DeleteView
```

```

from django.urls import reverse_lazy
from django.contrib.auth.mixins import LoginRequiredMixin, UserPassesTestMixin
from django.core.exceptions import PermissionDenied, ValidationError
from django.contrib.auth.password_validation import validate_password
from django.views.decorators.http import require_POST
from django.db import transaction

from .models import User, Role, RolePermission, PasswordResetRequest, PermissionChangeLog
from accounts.permissions import capabilities_by_module, capability_codenames
from .forms import (
    CustomAuthenticationForm, CustomUserCreationForm,
    CustomUserChangeForm, UserProfileForm
)

```

urls.py (first lines)

```

from django.urls import path
from . import views

app_name = 'accounts'

urlpatterns = [
    path('login/', views.CustomLoginView.as_view(), name='login'),
    path('logout/', views.CustomLogoutView.as_view(), name='logout'),
    path('profile/', views.profile_view, name='profile'),
    path('users/', views.UserListView.as_view(), name='user_list'),
    path('users/create/', views.UserCreateView.as_view(), name='user_create'),
    path('users//edit/', views.UserUpdateView.as_view(), name='user_edit'),
    path('users//delete/', views.UserDeleteView.as_view(), name='user_delete'),
    path('fix-admin-role/', views.fix_admin_role, name='fix_admin_role'),

    # Password Reset (admin-controlled)
    path('users//send-reset/', views.send_reset_link, name='send_reset_link'),
    path('users/send-reset-all/', views.send_reset_link_all, name='send_reset_all'),
    path('forgot-password/', views.forgot_password, name='forgot_password'),
    path('reset-password//', views.reset_password_form, name='reset_password'),

```

forms.py (first lines)

```

from django import forms
from django.contrib.auth.forms import UserCreationForm, UserChangeForm, AuthenticationForm
from .models import User, Role
from projects.models import Region

class CustomAuthenticationForm(AuthenticationForm):
    """Custom login form with Bootstrap styling"""
    username = forms.CharField(
        widget=forms.TextInput(attrs={
            'class': 'form-control',
            'placeholder': 'Username'
        })
    )
    password = forms.CharField(
        widget=forms.PasswordInput(attrs={
            'class': 'form-control',
            'placeholder': 'Password'
        })
    )

```

admin.py (first lines)

```

from django.contrib import admin
from django.contrib.auth.admin import UserAdmin as BaseUserAdmin
from .models import User, Role

@admin.register(Role)

```

```

class RoleAdmin(admin.ModelAdmin):
list_display = ['name', 'description', 'created_at']
search_fields = ['name', 'description']

@admin.register(User)
class UserAdmin(BaseUserAdmin):
list_display = ['username', 'email', 'first_name', 'last_name', 'role', 'region', 'is_active']
list_filter = ['role', 'region', 'is_active', 'is_staff']
search_fields = ['username', 'email', 'first_name', 'last_name']

fieldsets = BaseUserAdmin.fieldsets + (
('Additional Info', {
'fields': ('role', 'region', 'phone', 'employee_code')

```

tests.py (first lines)

```

import json
import secrets
from datetime import timedelta
from django.test import TestCase, RequestFactory
from django.db import IntegrityError
from django.core.exceptions import PermissionDenied
from django.http import HttpResponse
from django.template import Template, Context
from django.urls import reverse
from django.utils import timezone
from accounts.permissions import CAPABILITIES, Capability, capability_codenames, require_capability,
CapabilityRequiredMixin, seed_default_permissions, DEFAULT_MODULE_ACCESS
from accounts.models import Role, RolePermission, PermissionChangeLog, User, PasswordResetRequest
from accounts.forms import CustomUserCreationForm
from projects.models import Project, Region, ProjectStatus

```

```

class RegistryTests(TestCase):
def test_codenames_are_unique(self):
codes = [c.codename for c in CAPABILITIES]
self.assertEqual(len(codes), len(set(codes)), "duplicate capability codenames")

```

Quick troubleshooting hints:

- If a DB/schema bug: inspect C:\Users\Asadullah\OneDrive\Desktop\ERP\leap-erp\accounts\models.py and migrations/ if present.
- If an endpoint or view logic bug: inspect C:\Users\Asadullah\OneDrive\Desktop\ERP\leap-erp\accounts\views.py and C:\Users\Asadullah\OneDrive\Desktop\ERP\leap-erp\accounts\urls.py
- If a form validation or input issue: inspect C:\Users\Asadullah\OneDrive\Desktop\ERP\leap-erp\accounts\forms.py
- If an admin-only issue: inspect C:\Users\Asadullah\OneDrive\Desktop\ERP\leap-erp\accounts\admin.py

App: attendance

Path: C:\Users\Asadullah\OneDrive\Desktop\ERP\leap-erp\attendance

****Primary purpose (from docstring):**** Wi-Fi based automatic attendance.

An agent on each employee laptop pings the ERP every minute with the BSSID of the Wi-Fi it's connected to + how long the user has been idle. A ping only counts toward attendance when the laptop is on a *registered office AP*, the user is *active* (idle below the threshold), and it's *within work hours*. Counted pings roll up into a per-employee-per-day AttendanceDay.

Files:

- README.md
- __init__.py
- admin.py
- api.py

- apps.py
- forms.py
- models.py
- services.py
- tests.py
- ui_urls.py
- urls.py
- views.py

Key places to look when troubleshooting:

- models.py — database schema and model methods (data validation/business rules tied to persistence)
- views.py — request handling, business workflows, API endpoints and templates rendered
- forms.py — input validation and form-level logic
- urls.py — routing (which endpoint maps to which view)
- admin.py — admin UI customisations (if the bug appears in Django admin)

models.py (first lines)

```
"""Wi-Fi based automatic attendance.
```

An agent on each employee laptop pings the ERP every minute with the BSSID of the Wi-Fi it's connected to + how long the user has been idle. A ping only counts toward attendance when the laptop is on a *registered office AP*, the user is *active* (idle below the threshold), and it's *within work hours*.

Counted pings roll up into a per-employee-per-day AttendanceDay.

```
"""
```

```
import secrets
```

```
from django.db import models
```

```
def _gen_token():
```

```
    return secrets.token_urlsafe(32)
```

```
class OfficeNetwork(models.Model):
```

```
    """A registered office access point. A heartbeat only counts when the
    laptop is connected to one of these BSSIDs — so being on a phone hotspot
```

views.py (first lines)

```
"""In-app registration UI for Wi-Fi attendance — office access points and
employee devices — plus a token-map CSV export for provisioning agents.
Admin / super-admin only (registration is an IT/HR-admin task)."""
```

```
import csv
```

```
from django.contrib import messages
```

```
from django.contrib.auth.decorators import login_required
```

```
from django.http import HttpResponse
```

```
from django.shortcuts import get_object_or_404, redirect, render
```

```
from .forms import OfficeNetworkForm, RegisteredDeviceForm
```

```
from .models import OfficeNetwork, RegisteredDevice
```

```
def _can_manage(user):
```

```
    return bool(getattr(user, 'is_authenticated', False)
```

```
    and (user.is_superuser or user.is_admin_user))
```

```
@login_required
```

urls.py (first lines)

```
from django.urls import path
```

```
from . import api
```

```
app_name = 'attendance'
```

```
urlpatterns = [
```

```
    path('checkin/', api.checkin, name='checkin'),
```

```
]
```

forms.py (first lines)

```
import re
from django import forms
from hr.models import Employee
from .models import OfficeNetwork, RegisteredDevice
_BSSID_RE = re.compile(r'^([0-9a-f]{2}:){5}[0-9a-f]{2}$')

class OfficeNetworkForm(forms.ModelForm):
    class Meta:
        model = OfficeNetwork
        fields = ['bssid', 'label']
        widgets = {
            'bssid': forms.TextInput(attrs={'class': 'form-control form-control-sm',
            'placeholder': 'a1:b2:c3:d4:e5:f6'}),
            'label': forms.TextInput(attrs={'class': 'form-control form-control-sm',
            'placeholder': 'HO 2nd-floor AP'}),
        }
```

admin.py (first lines)

```
from django.contrib import admin
from .models import OfficeNetwork, RegisteredDevice, Heartbeat, AttendanceDay

@admin.register(OfficeNetwork)
class OfficeNetworkAdmin(admin.ModelAdmin):
    list_display = ('label', 'bssid', 'is_active', 'created_at')
    list_filter = ('is_active',)
    search_fields = ('label', 'bssid')

@admin.register(RegisteredDevice)
class RegisteredDeviceAdmin(admin.ModelAdmin):
    list_display = ('employee', 'label', 'serial_number', 'token', 'is_active', 'first_seen_at', 'last_seen_at')
    list_filter = ('is_active',)
    search_fields = ('employee__full_name', 'employee__iqama_number', 'label', 'serial_number', 'token')
    readonly_fields = ('token', 'first_seen_at', 'last_seen_at', 'created_at')
    autocomplete_fields = ('employee', 'asset')
```

tests.py (first lines)

```
import json
from datetime import time

from django.test import TestCase, override_settings
from django.urls import reverse
from django.utils import timezone

from hr.models import Employee
from attendance.models import OfficeNetwork, RegisteredDevice, Heartbeat, AttendanceDay
OFFICE_BSSID = 'a1:b2:c3:d4:e5:f6'

def _wide_hours():
    # Work window that always contains "now" so tests aren't clock-dependent.
    return override_settings(ATT_WORK_START='00:00', ATT_WORK_END='23:59',
    ATT_MAX_IDLE_SECONDS=300, ATT_MIN_MINUTES_PRESENT=1)

class CheckInTests(TestCase):
```

Quick troubleshooting hints:

- If a DB/schema bug: inspect C:\Users\Asadullah\OneDrive\Desktop\ERP\leap-erp\attendance\models.py and migrations/ if present.
- If an endpoint or view logic bug: inspect C:\Users\Asadullah\OneDrive\Desktop\ERP\leap-erp\attendance\views.py and C:\Users\Asadullah\OneDrive\Desktop\ERP\leap-erp\attendance\urls.py

- If a form validation or input issue: inspect
C:\Users\Asadullah\OneDrive\Desktop\ERP\leap-erp\attendance\forms.py
- If an admin-only issue: inspect C:\Users\Asadullah\OneDrive\Desktop\ERP\leap-erp\attendance\admin.py

App: company

Path: C:\Users\Asadullah\OneDrive\Desktop\ERP\leap-erp\company
Primary purpose: (No module docstring found; inferred from filenames below)

Files:

- __init__.py
- admin.py
- apps.py
- forms.py
- models.py
- tests.py
- urls.py
- views.py

Key places to look when troubleshooting:

- models.py — database schema and model methods (data validation/business rules tied to persistence)
- views.py — request handling, business workflows, API endpoints and templates rendered
- forms.py — input validation and form-level logic
- urls.py — routing (which endpoint maps to which view)
- admin.py — admin UI customisations (if the bug appears in Django admin)

models.py (first lines)

```
from django.conf import settings
from django.db import models
```

```
class CompanyDocument(models.Model):
    """Company-wide important documents: certifications, registrations,
    licenses, ISO certificates, legal/financial documents, etc."""
```

```
DOC_TYPE_CHOICES = [
    ('certification', 'Certification'),
    ('registration', 'Registration'),
    ('license', 'License / Permit'),
    ('iso', 'ISO Certificate'),
    ('legal', 'Legal Document'),
    ('financial', 'Financial Document'),
    ('insurance', 'Insurance'),
    ('other', 'Other'),
]
```

```
title = models.CharField(max_length=255)
```

views.py (first lines)

```
from django.contrib import messages
from django.contrib.auth.decorators import login_required
from django.contrib.auth.mixins import LoginRequiredMixin, UserPassesTestMixin
from django.db.models import Q
from django.shortcuts import get_object_or_404, redirect, render
from django.views.generic import ListView

from .forms import CompanyDocumentForm
from .models import CompanyDocument
```

```
def _is_company_admin(user):
    return user.is_authenticated and (
        user.is_superuser or user.is_admin_user)
```

```

class CompanyDocumentListView(LoginRequiredMixin, UserPassesTestMixin, ListView):
    model = CompanyDocument
    template_name = 'company/company_document_list.html'
    context_object_name = 'documents'

```

urls.py (first lines)

```

from django.urls import path
from . import views
app_name = 'company'
urlpatterns = [
    path('documents/', views.CompanyDocumentListView.as_view(), name='document_list'),
    path('documents/upload/', views.company_document_upload, name='document_upload'),
    path('documents/edit/', views.company_document_edit, name='document_edit'),
    path('documents/delete/', views.company_document_delete, name='document_delete'),
]

```

forms.py (first lines)

```

from django import forms
from .models import CompanyDocument

class CompanyDocumentForm(forms.ModelForm):
    class Meta:
        model = CompanyDocument
        fields = ['title', 'document_type', 'custom_type', 'file',
                  'issuing_authority', 'reference_number',
                  'issue_date', 'expiry_date', 'notes']
        widgets = {
            'issue_date': forms.DateInput(attrs={'type': 'date'}),
            'expiry_date': forms.DateInput(attrs={'type': 'date'}),
            'notes': forms.Textarea(attrs={'rows': 2}),
            'custom_type': forms.TextInput(
                attrs={'placeholder': 'Type a label (used when "Other")'}),
        }
    def __init__(self, *args, **kwargs):

```

admin.py (first lines)

```

from django.contrib import admin
from .models import CompanyDocument

@admin.register(CompanyDocument)
class CompanyDocumentAdmin(admin.ModelAdmin):
    list_display = ('title', 'document_type', 'reference_number',
                    'issue_date', 'expiry_date', 'uploaded_at')
    list_filter = ('document_type',)
    search_fields = ('title', 'issuing_authority', 'reference_number')

```

tests.py (first lines)

```

import tempfile

from django.core.files.storage import default_storage
from django.core.files.uploadedfile import SimpleUploadedFile
from django.test import TestCase, override_settings
from django.urls import reverse

from accounts.models import Role, User
from company.models import CompanyDocument

@override_settings(MEDIA_ROOT=tempfile.mkdtemp())
class CompanyDocumentTests(TestCase):
    def setUp(self):

```

```
sa, _ = Role.objects.get_or_create(name=Role.SUPER_ADMIN)
self.admin = User.objects.create_user('admin1', password='x')
self.admin.role = sa
self.admin.save()
self.plain = User.objects.create_user('plain', password='x')
```

Quick troubleshooting hints:

- If a DB/schema bug: inspect C:\Users\Asadullah\OneDrive\Desktop\ERP\leap-erp\company\models.py and migrations/ if present.
- If an endpoint or view logic bug: inspect C:\Users\Asadullah\OneDrive\Desktop\ERP\leap-erp\company\views.py and C:\Users\Asadullah\OneDrive\Desktop\ERP\leap-erp\company\urls.py
- If a form validation or input issue: inspect C:\Users\Asadullah\OneDrive\Desktop\ERP\leap-erp\company\forms.py
- If an admin-only issue: inspect C:\Users\Asadullah\OneDrive\Desktop\ERP\leap-erp\company\admin.py

App: contacts

Path: C:\Users\Asadullah\OneDrive\Desktop\ERP\leap-erp\contacts

****Primary purpose:**** (No module docstring found; inferred from filenames below)

Files:

- __init__.py
- admin.py
- apps.py
- forms.py
- models.py
- signals.py
- tests.py
- urls.py
- views.py

Key places to look when troubleshooting:

- models.py — database schema and model methods (data validation/business rules tied to persistence)
- views.py — request handling, business workflows, API endpoints and templates rendered
- forms.py — input validation and form-level logic
- urls.py — routing (which endpoint maps to which view)
- admin.py — admin UI customisations (if the bug appears in Django admin)

models.py (first lines)

```
from django.db import models
from django.conf import settings
```

```
class ContactDatabase(models.Model):
```

```
    """
```

Centralized Contact Database for all technology categories.

Stores leads and contacts from CCTV, Radios, ACS, IoT, IIoT, Servers, Network & Security, Firewall, Cyber Security, Windows, OT categories.

```
    """
```

```
CATEGORY_CHOICES = [
    ('cctv', 'CCTV'),
    ('radios', 'Radios'),
    ('acs', 'Access Control Systems (ACS)'),
    ('iot', 'IoT'),
    ('iiot', 'IIoT'),
    ('servers', 'Servers'),
    ('network_security', 'Network & Security'),
    ('firewall', 'Firewall'),
```


views.py (first lines)

```
from django.shortcuts import render, redirect
from django.contrib.auth.mixins import LoginRequiredMixin
from django.contrib.auth.decorators import login_required
from django.contrib import messages
from django.views.generic import ListView, DetailView, CreateView, UpdateView, DeleteView
from django.urls import reverse_lazy
from django.db.models import Q, Count
from django.http import HttpResponse, JsonResponse
from datetime import datetime
import openpyxl
from openpyxl.styles import Font, Alignment, PatternFill, Border, Side

from .models import ContactDatabase
from .forms import ContactDatabaseForm, ContactDatabaseFilterForm, ContactImportForm
from notifications.services import notify_users

class ContactListView(LoginRequiredMixin, ListView):
    """List all contacts with filtering by category."""
    model = ContactDatabase
```

urls.py (first lines)

```
from django.urls import path
from . import views

app_name = 'contacts'

urlpatterns = [
    # Main list view
    path("", views.ContactListView.as_view(), name='contact_list'),
    # Category-specific views
    path('category/', views.ContactByCategoryView.as_view(), name='contact_by_category'),
    # CRUD operations
    path('add/', views.ContactCreateView.as_view(), name='contact_create'),
    path('/', views.ContactDetailView.as_view(), name='contact_detail'),
    path('/edit/', views.ContactUpdateView.as_view(), name='contact_update'),
    path('/delete/', views.ContactDeleteView.as_view(), name='contact_delete'),
    # Import/Export
    path('import/', views.contact_import, name='contact_import'),
```

forms.py (first lines)

```
from django import forms
from .models import ContactDatabase

class ContactDatabaseForm(forms.ModelForm):
    """Form for creating and editing contacts in the database."""

    class Meta:
        model = ContactDatabase
        fields = [
            'category', 'notice_identifier', 'notice_type', 'serial_number',
            'status', 'published_date', 'organisation_name', 'title',
            'description', 'nationwide', 'postcode', 'region', 'contact_name',
            'contact_email', 'contact_address', 'contact_telephone',
            'contact_website', 'cpv_codes', 'last_contact', 'comments'
        ]
        widgets = {
            'category': forms.Select(attrs={'class': 'form-select'}),
            'notice_identifier': forms.TextInput(attrs={'class': 'form-control'}),
            'notice_type': forms.Select(attrs={'class': 'form-select'}),
```

admin.py (first lines)

```
from django.contrib import admin
```

Register your models here.

signals.py (first lines)

```
import logging

from django.db.models.signals import post_save
from django.dispatch import receiver

from reports.models import SalesCallReport
from .models import ContactDatabase

logger = logging.getLogger(__name__)
```

Categories shared between SalesCallReport and ContactDatabase

```
SHARED_CATEGORIES = {
    'cctv', 'radios', 'acs', 'iot', 'iiot', 'servers',
    'network_security', 'firewall', 'cyber_security', 'windows', 'ot',
}

def _map_category(report):
    """Extract first matching category from the sales call's system_categories."""
    selected = report.get_system_categories_list()
```

tests.py (first lines)

```
from django.test import TestCase
```

Create your tests here.

Quick troubleshooting hints:

- If a DB/schema bug: inspect C:\Users\Asadullah\OneDrive\Desktop\ERP\leap-erp\contacts\models.py and migrations/ if present.
 - If an endpoint or view logic bug: inspect C:\Users\Asadullah\OneDrive\Desktop\ERP\leap-erp\contacts\views.py and C:\Users\Asadullah\OneDrive\Desktop\ERP\leap-erp\contacts\urls.py
 - If a form validation or input issue: inspect C:\Users\Asadullah\OneDrive\Desktop\ERP\leap-erp\contacts\forms.py
 - If an admin-only issue: inspect C:\Users\Asadullah\OneDrive\Desktop\ERP\leap-erp\contacts\admin.py
-

App: costing

Path: C:\Users\Asadullah\OneDrive\Desktop\ERP\leap-erp\costing

****Primary purpose:**** (No module docstring found; inferred from filenames below)

Files:

- __init__.py
- admin.py
- apps.py
- forms.py
- models.py
- tests.py
- urls.py
- views.py

Key places to look when troubleshooting:

- models.py — database schema and model methods (data validation/business rules tied to persistence)
- views.py — request handling, business workflows, API endpoints and templates rendered
- forms.py — input validation and form-level logic
- urls.py — routing (which endpoint maps to which view)

- admin.py — admin UI customisations (if the bug appears in Django admin)

models.py (first lines)

```
from django.db import models
from django.conf import settings
from decimal import Decimal
from datetime import timedelta

def working_days_between(start, end):
    """Whole working days from `start` to `end`, excluding the KSA Fri/Sat
    weekend. Returns None if either bound is missing, 0 if same day / end<=start.

    Both bounds are aware datetimes; only the local calendar date matters.
    """
    if not start or not end:
        return None
    from django.utils import timezone
    s = timezone.localtime(start).date()
    e = timezone.localtime(end).date()
    if e <= s:
        return 0
    days, d = 0, s
```

views.py (first lines)

```
from django.views.generic import ListView, DetailView, CreateView, UpdateView, DeleteView
from django.contrib.auth.mixins import LoginRequiredMixin, UserPassesTestMixin
from django.shortcuts import get_object_or_404, redirect, render
from django.urls import reverse_lazy, reverse
from django.contrib import messages
from django.http import HttpResponse, JsonResponse
from django.views.decorators.http import require_POST
from django.contrib.auth.decorators import login_required
from django.db import transaction
from django.db.models import Q
from django.template.loader import render_to_string
from django.utils.http import url_has_allowed_host_and_scheme
from django.utils.text import slugify
from django.utils import timezone
from decimal import Decimal, InvalidOperation
import logging
from accounts.permissions import CapabilityRequiredMixin
logger = logging.getLogger(__name__)
```

urls.py (first lines)

```
from django.urls import path
from django.contrib.auth.decorators import login_required
from . import views

app_name = 'costing'

urlpatterns = [
    # Costing sheets
    path("", views.CostingListView.as_view(), name='list'),
    path('pipeline-pdf/', login_required(views.costing_pipeline_pdf), name='pipeline_pdf'),
    path('pipeline-excel/', login_required(views.costing_pipeline_excel), name='pipeline_excel'),
    path('create/', views.CostingCreateView.as_view(), name='create'),
    path('import/', login_required(views.costing_import_new), name='import_new'),
    path('/', views.CostingDetailView.as_view(), name='detail'),
    path('/edit/', views.CostingUpdateView.as_view(), name='edit'),
    path('/delete/', views.CostingDeleteView.as_view(), name='delete'),
    path('/export/', login_required(views.costing_export_excel), name='export'),
```

```

path('/export-pdf/', login_required(views.costing_export_pdf), name='export_pdf'),
path('/margin-analysis/', views.costing_margin_analysis, name='margin_analysis'),
path('/import/', login_required(views.costing_import_excel), name='import_excel'),

```

forms.py (first lines)

```

from django import forms
from .models import ExchangeRate, CostingSheet, CostingSection, CostingLineItem, TermsTemplate,
ScopeOfWorkItem, ClientRemarkTemplate, ClientRemarkPair
from projects.models import Project

class CostingSheetForm(forms.ModelForm):
    class Meta:
        model = CostingSheet
        fields = [
            'title', 'project', 'customer_reference',
            'margin', 'discount_rate', 'shipping_rate', 'customs_rate',
            'finances_rate', 'installation_rate',
            'output_currency', 'status',
            'customer_name', 'end_user', 'contact_person', 'telephone', 'fax',
        ]
        widgets = {
            'margin': forms.NumberInput(attrs={'step': '0.01', 'min': '0', 'max': '99', 'placeholder': 'e.g. 40 for 40% (max 99)'}),
            'discount_rate': forms.NumberInput(attrs={'step': '0.01', 'min': '0', 'max': '100', 'placeholder': 'e.g. 5 for 5%'}),
            'shipping_rate': forms.NumberInput(attrs={'step': '0.01', 'min': '0', 'max': '100', 'placeholder': 'e.g. 3 for 3%'}),
            'customs_rate': forms.NumberInput(attrs={'step': '0.01', 'min': '0', 'max': '100', 'placeholder': 'e.g. 5 for 5%'}),

```

admin.py (first lines)

```

from django.contrib import admin
from .models import ExchangeRate, CostingSheet, CostingSection, CostingLineItem

@admin.register(ExchangeRate)
class ExchangeRateAdmin(admin.ModelAdmin):
    list_display = ['currency_code', 'currency_name', 'rate_to_usd', 'updated_at']
    search_fields = ['currency_code', 'currency_name']

class CostingSectionInline(admin.TabularInline):
    model = CostingSection
    extra = 0

@admin.register(CostingSheet)
class CostingSheetAdmin(admin.ModelAdmin):
    list_display = ['title', 'project', 'status', 'margin', 'output_currency', 'created_by', 'updated_at']
    list_filter = ['status', 'output_currency']
    search_fields = ['title', 'customer_reference']

```

tests.py (first lines)

```

import tempfile
from decimal import Decimal

from django.test import TestCase, override_settings
from django.urls import reverse

from accounts.models import Role, User
from projects.models import Region, ProjectStatus, Project
from costing.models import CostingSheet
from django.test import TestCase, Client
from accounts.models import User

class CostingExcelExportLinkTests(TestCase):
    """BUG-005: the Excel export link must not use target="_blank",
    since a file download response leaves an empty tab open."""

    def setUp(self):
        self.client = Client()

```

```
from accounts.models import Role
```

Quick troubleshooting hints:

- If a DB/schema bug: inspect C:\Users\Asadullah\OneDrive\Desktop\ERP\leap-erp\costing\models.py and migrations/ if present.
- If an endpoint or view logic bug: inspect C:\Users\Asadullah\OneDrive\Desktop\ERP\leap-erp\costing\views.py and C:\Users\Asadullah\OneDrive\Desktop\ERP\leap-erp\costing\urls.py
- If a form validation or input issue: inspect C:\Users\Asadullah\OneDrive\Desktop\ERP\leap-erp\costing\forms.py
- If an admin-only issue: inspect C:\Users\Asadullah\OneDrive\Desktop\ERP\leap-erp\costing\admin.py

App: dashboard

Path: C:\Users\Asadullah\OneDrive\Desktop\ERP\leap-erp\dashboard

****Primary purpose:**** (No module docstring found; inferred from filenames below)

Files:

- __init__.py
- admin.py
- apps.py
- models.py
- my_work.py
- search.py
- storage_cleanup.py
- storage_report.py
- tests.py
- urls.py
- views.py

Key places to look when troubleshooting:

- models.py — database schema and model methods (data validation/business rules tied to persistence)
- views.py — request handling, business workflows, API endpoints and templates rendered
- forms.py — input validation and form-level logic
- urls.py — routing (which endpoint maps to which view)
- admin.py — admin UI customisations (if the bug appears in Django admin)

models.py (first lines)

```
from django.db import models
```

Create your models here.

views.py (first lines)

```
from django.shortcuts import render
from django.contrib.auth.decorators import login_required
from django.db.models import Sum, Count, Q
from django.http import JsonResponse
from accounts.permissions import require_capability
from projects.models import Project, Region, ProjectStatus
from accounts.models import User

def get_region_stats(projects, region_codes):
    """Helper to get stats for a specific region or regions"""
    region_projects = projects.filter(region__code__in=region_codes)
    active = region_projects.filter(status__category='active')
    hot_leads = region_projects.filter(status__category='hot_lead')
    won = region_projects.filter(status__category='won')
    lost = region_projects.filter(status__category='lost')
    ongoing = region_projects.filter(status__category='ongoing')
```

urls.py (first lines)

```
from django.urls import path
from . import views
from .search import global_search
from .my_work import my_work
app_name = 'dashboard'
urlpatterns = [
    path("", views.index, name='index'),
    path('my-work/', my_work, name='my_work'),
    path('api/chart-data/', views.chart_data, name='chart_data'),
    path('api/search/', global_search, name='global_search'),
    path('storage/', views.storage_report, name='storage_report'),
    path('storage/preview/', views.storage_orphan_preview, name='storage_orphan_preview'),
]
```

admin.py (first lines)

```
from django.contrib import admin
```

Register your models here.

tests.py (first lines)

```
import tempfile

from django.core.files.base import ContentFile
from django.core.files.storage import default_storage
from django.test import TestCase, override_settings
from django.urls import reverse

from accounts.models import Role, User

@override_settings(MEDIA_ROOT=tempfile.mkdtemp())
class StorageReportTests(TestCase):
    """The Storage admin page and the orphan-preview endpoint are super-admin
    only; preview redirects to an existing object and 404s otherwise."""

    def setUp(self):
        sa, _ = Role.objects.get_or_create(name=Role.SUPER_ADMIN)
        self.super = User.objects.create_user('sa', password='x')
        self.super.role = sa
        self.super.save()
```

Quick troubleshooting hints:

- If a DB/schema bug: inspect C:\Users\Asadullah\OneDrive\Desktop\ERP\leap-erp\dashboard\models.py and migrations/ if present.
- If an endpoint or view logic bug: inspect C:\Users\Asadullah\OneDrive\Desktop\ERP\leap-erp\dashboard\views.py and C:\Users\Asadullah\OneDrive\Desktop\ERP\leap-erp\dashboard\urls.py
- If a form validation or input issue: inspect C:\Users\Asadullah\OneDrive\Desktop\ERP\leap-erp\dashboard\forms.py
- If an admin-only issue: inspect C:\Users\Asadullah\OneDrive\Desktop\ERP\leap-erp\dashboard\admin.py

App: devtracking

Path: C:\Users\Asadullah\OneDrive\Desktop\ERP\leap-erp\devtracking

****Primary purpose:**** (No module docstring found; inferred from filenames below)

Files:

- __init__.py
- admin.py
- ai.py
- apps.py

- forms.py
- github.py
- models.py
- tests.py
- urls.py
- views.py

Key places to look when troubleshooting:

- models.py — database schema and model methods (data validation/business rules tied to persistence)
- views.py — request handling, business workflows, API endpoints and templates rendered
- forms.py — input validation and form-level logic
- urls.py — routing (which endpoint maps to which view)
- admin.py — admin UI customisations (if the bug appears in Django admin)
- templates/ — HTML templates for UI (check for presentation/JS bugs)

models.py (first lines)

```
from django.db import models
from django.conf import settings
from django.utils import timezone

class TaskStack(models.Model):
    name = models.CharField(max_length=200)
    description = models.TextField(blank=True)
    created_by = models.ForeignKey(settings.AUTH_USER_MODEL, on_delete=models.SET_NULL,
    null=True, blank=True, related_name='dev_stacks_created')
    created_at = models.DateTimeField(auto_now_add=True)

    class Meta:
        ordering = ['-created_at']

    def __str__(self):
        return self.name

class DevTask(models.Model):
```

views.py (first lines)

```
from datetime import timedelta

from django.http import HttpResponseForbidden, HttpResponseBadRequest
from django.shortcuts import get_object_or_404, redirect, render
from django.urls import reverse, reverse_lazy
from django.utils import timezone
from django.views.decorators.http import require_POST
from django.views.generic import (TemplateView, CreateView, UpdateView,
DeleteView, ListView, DetailView)

from accounts.models import AI_DEVELOPER_ROLE_NAMES, User
from accounts.permissions import CapabilityRequiredMixin, require_capability
from notifications.services import notify_users

from .forms import DevTaskForm, BulkTaskForm, AssignTaskForm, StackForm
from .models import DevTask, DevTaskUpdate, DevDigest, TaskStack

class DashboardView(CapabilityRequiredMixin, TemplateView):
    capability = 'devtracking.admin'
```

urls.py (first lines)

```
from django.urls import path

from . import views

app_name = 'devtracking'

urlpatterns = [
    path("", views.DashboardView.as_view(), name='dashboard'),
    path('assign/', views.TaskAssignView.as_view(), name='assign'),
```

```

path('tasks/bulk/', views.bulk_create, name='bulk_create'),
path('backlog/', views.BacklogListView.as_view(), name='backlog'),
path('tasks//assign/', views.assign_existing, name='assign_existing'),
path('tasks//edit/', views.TaskEditView.as_view(), name='task_edit'),
path('tasks//delete/', views.TaskDeleteView.as_view(), name='task_delete'),
path('tasks/', views.TaskListView.as_view(), name='tasks'),
path('developer//', views.DevDetailView.as_view(), name='dev_detail'),
path('my-tasks/', views.MyTasksView.as_view(), name='my_tasks'),
path('tasks//action/', views.task_action, name='task_action'),
path('tasks//refresh-github/', views.refresh_github, name='refresh_github'),
path('generate/', views.generate_now, name='generate_now'),

```

forms.py (first lines)

```

from django import forms
from accounts.models import AI_DEVELOPER_ROLE_NAMES, User
from .models import DevTask, TaskStack

class DevTaskForm(forms.ModelForm):
    class Meta:
        model = DevTask
        fields = ['developer', 'title', 'description', 'priority',
                  'estimated_hours', 'due_date', 'github_url', 'stack']
        widgets = {
            'developer': forms.Select(attrs={'class': 'form-select'}),
            'title': forms.TextInput(attrs={'class': 'form-control'}),
            'description': forms.Textarea(attrs={'class': 'form-control', 'rows': 3}),
            'priority': forms.Select(attrs={'class': 'form-select'}),
            'estimated_hours': forms.NumberInput(attrs={'class': 'form-control', 'step': '0.5'}),
            'due_date': forms.DateInput(attrs={'class': 'form-control', 'type': 'date'}),
            'github_url': forms.URLInput(attrs={'class': 'form-control', 'placeholder': 'Optional PR/branch link'}),
            'stack': forms.Select(attrs={'class': 'form-select'}),
        }

```

admin.py (first lines)

```

from django.contrib import admin
from .models import DevTask, DevTaskUpdate, DevDigest, TaskStack

@admin.register(DevTask)
class DevTaskAdmin(admin.ModelAdmin):
    list_display = ['title', 'developer', 'status', 'priority', 'due_date', 'completed_at']
    list_filter = ['status', 'priority']
    search_fields = ['title', 'developer__username']

@admin.register(TaskStack)
class TaskStackAdmin(admin.ModelAdmin):
    list_display = ['name', 'created_by', 'created_at']
    search_fields = ['name']

admin.site.register(DevTaskUpdate)
admin.site.register(DevDigest)

```

tests.py (first lines)

```

from datetime import date, timedelta
from django.test import TestCase
from django.utils import timezone
from accounts.models import Role, User

def mkuser(username, role_name):
    role, _ = Role.objects.get_or_create(name=role_name)
    u = User.objects.create_user(username, password='x'); u.role = role; u.save()
    return u

```



```
class DevTaskModelTests(TestCase):
def setUp(self):
self.admin = mkuser('adm', Role.ADMIN)
self.dev = mkuser('dev', Role.DEVELOPER)

def _task(self, **kw):
from devtracking.models import DevTask
kw.setdefault('title', 'T'); kw.setdefault('developer', self.dev)
Relationships referenced (from scanned first lines):
- FK -> settings.AUTH_USER_MODEL
```

Quick troubleshooting hints:

- If a DB/schema bug: inspect C:\Users\Asadullah\OneDrive\Desktop\ERP\leap-erp\devtracking\models.py and migrations/ if present.
- If an endpoint or view logic bug: inspect C:\Users\Asadullah\OneDrive\Desktop\ERP\leap-erp\devtracking\views.py and C:\Users\Asadullah\OneDrive\Desktop\ERP\leap-erp\devtracking\urls.py
- If a form validation or input issue: inspect C:\Users\Asadullah\OneDrive\Desktop\ERP\leap-erp\devtracking\forms.py
- If an admin-only issue: inspect C:\Users\Asadullah\OneDrive\Desktop\ERP\leap-erp\devtracking\admin.py

App: docs

Path: C:\Users\Asadullah\OneDrive\Desktop\ERP\leap-erp\docs

****Primary purpose:**** (No module docstring found; inferred from filenames below)

Files:

Key places to look when troubleshooting:

- models.py — database schema and model methods (data validation/business rules tied to persistence)
- views.py — request handling, business workflows, API endpoints and templates rendered
- forms.py — input validation and form-level logic
- urls.py — routing (which endpoint maps to which view)
- admin.py — admin UI customisations (if the bug appears in Django admin)

Quick troubleshooting hints:

- If a DB/schema bug: inspect C:\Users\Asadullah\OneDrive\Desktop\ERP\leap-erp\docs\models.py and migrations/ if present.
- If an endpoint or view logic bug: inspect C:\Users\Asadullah\OneDrive\Desktop\ERP\leap-erp\docs\views.py and C:\Users\Asadullah\OneDrive\Desktop\ERP\leap-erp\docs\urls.py
- If a form validation or input issue: inspect C:\Users\Asadullah\OneDrive\Desktop\ERP\leap-erp\docs\forms.py
- If an admin-only issue: inspect C:\Users\Asadullah\OneDrive\Desktop\ERP\leap-erp\docs\admin.py

App: drafts

Path: C:\Users\Asadullah\OneDrive\Desktop\ERP\leap-erp\drafts

****Primary purpose:**** (No module docstring found; inferred from filenames below)

Files:

- __init__.py
- admin.py
- apps.py
- models.py
- registry.py
- tests.py
- urls.py
- views.py

Key places to look when troubleshooting:

- models.py — database schema and model methods (data validation/business rules tied to persistence)

- views.py — request handling, business workflows, API endpoints and templates rendered
- forms.py — input validation and form-level logic
- urls.py — routing (which endpoint maps to which view)
- admin.py — admin UI customisations (if the bug appears in Django admin)

models.py (first lines)

```
from django.db import models
from django.conf import settings
from django.contrib.contenttypes.fields import GenericForeignKey
from django.contrib.contenttypes.models import ContentType

class FormDraft(models.Model):
    user = models.ForeignKey(
        settings.AUTH_USER_MODEL,
        on_delete=models.CASCADE,
        related_name='form_drafts',
    )
    form_key = models.CharField(max_length=64)
    content_type = models.ForeignKey(ContentType, null=True, blank=True, on_delete=models.SET_NULL)
    object_id = models.PositiveIntegerField(null=True, blank=True)
    target = GenericForeignKey('content_type', 'object_id')
    data = models.JSONField()
    updated_at = models.DateTimeField(auto_now=True)
    created_at = models.DateTimeField(auto_now_add=True)
```

views.py (first lines)

```
import json
from django.contrib.auth.decorators import login_required
from django.http import JsonResponse
from django.views.decorators.http import require_POST
from django.shortcuts import get_object_or_404
from django.utils.module_loading import import_string
from django.urls import reverse
import hmac
from django.middleware.csrf import get_token
from django.views.decorators.csrf import csrf_exempt

from .models import FormDraft
from .registry import FORM_REGISTRY
MAX_PAYLOAD_CHARS = 50000

@login_required
@require_POST
@csrf_exempt
def save_draft(request):
```

urls.py (first lines)

```
from django.urls import path
from . import views

app_name = 'drafts'

urlpatterns = [
    path('save/', views.save_draft, name='save'),
    path('check/', views.check_drafts, name='check'),
    path('/discard/', views.discard_draft, name='discard'),
]
```

admin.py (first lines)

```
from django.contrib import admin
```

Register your models here.

tests.py (first lines)

```
from django.test import TestCase
```

Create your tests here.

Relationships referenced (from scanned first lines):

- FK -> settings.AUTH_USER_MODEL
- FK -> ContentType
- FK -> content_type

Quick troubleshooting hints:

- If a DB/schema bug: inspect C:\Users\Asadullah\OneDrive\Desktop\ERP\leap-erp\drafts\models.py and migrations/ if present.
- If an endpoint or view logic bug: inspect C:\Users\Asadullah\OneDrive\Desktop\ERP\leap-erp\drafts\views.py and C:\Users\Asadullah\OneDrive\Desktop\ERP\leap-erp\drafts\urls.py
- If a form validation or input issue: inspect C:\Users\Asadullah\OneDrive\Desktop\ERP\leap-erp\drafts\forms.py
- If an admin-only issue: inspect C:\Users\Asadullah\OneDrive\Desktop\ERP\leap-erp\drafts\admin.py

App: `erp_leap`

Path: C:\Users\Asadullah\OneDrive\Desktop\ERP\leap-erp\erp_leap

****Primary purpose:**** (No module docstring found; inferred from filenames below)

Files:

- `__init__.py`
- `asgi.py`
- `settings.py`
- `urls.py`
- `wsgi.py`

Key places to look when troubleshooting:

- `models.py` — database schema and model methods (data validation/business rules tied to persistence)
- `views.py` — request handling, business workflows, API endpoints and templates rendered
- `forms.py` — input validation and form-level logic
- `urls.py` — routing (which endpoint maps to which view)
- `admin.py` — admin UI customisations (if the bug appears in Django admin)

urls.py (first lines)

.....

URL configuration for Leap Networks ERP project.

.....

```
from django.contrib import admin
from django.contrib.auth.decorators import login_required
from django.urls import path, include, re_path
from django.conf import settings
from django.conf.urls.static import static
from django.views.static import serve as static_serve

urlpatterns = [
    path('admin/', admin.site.urls),
    path("", include('dashboard.urls')),
    path('accounts/', include('accounts.urls')),
    path('projects/', include('projects.urls')),
    path('reports/', include('reports.urls')),
    path('database/', include('contacts.urls')),
    path('costing/', include('costing.urls')),
    path('notifications/', include('notifications.urls')),
```

Quick troubleshooting hints:

- If a DB/schema bug: inspect C:\Users\Asadullah\OneDrive\Desktop\ERP\leap-erp\erp_leap\models.py and migrations/ if present.
- If an endpoint or view logic bug: inspect C:\Users\Asadullah\OneDrive\Desktop\ERP\leap-erp\erp_leap\views.py and C:\Users\Asadullah\OneDrive\Desktop\ERP\leap-erp\erp_leap\urls.py
- If a form validation or input issue: inspect C:\Users\Asadullah\OneDrive\Desktop\ERP\leap-erp\erp_leap\forms.py
- If an admin-only issue: inspect C:\Users\Asadullah\OneDrive\Desktop\ERP\leap-erp\erp_leap\admin.py

App: finance

Path: C:\Users\Asadullah\OneDrive\Desktop\ERP\leap-erp\finance

****Primary purpose:**** (No module docstring found; inferred from filenames below)

Files:

- __init__.py
- admin.py
- apps.py
- models.py
- tests.py
- urls.py
- views.py

Key places to look when troubleshooting:

- models.py — database schema and model methods (data validation/business rules tied to persistence)
- views.py — request handling, business workflows, API endpoints and templates rendered
- forms.py — input validation and form-level logic
- urls.py — routing (which endpoint maps to which view)
- admin.py — admin UI customisations (if the bug appears in Django admin)

models.py (first lines)

```
from datetime import timedelta
from decimal import Decimal

from django.conf import settings
from django.db import models
```

```
class ProjectFinance(models.Model):
```

```
    """Finance workflow record for a Won project — Step 2 of finance approval.
```

```
    Holds the approved Project P.O Value (carried over from the approved margin
    scenario in Step 1) and the kickoff date that drives the milestone schedule.
    One per project.
    """
```

```
MARGIN_CHOICES = [
    ('M1', 'M1 — Current'),
    ('M2', 'M2 — High'),
    ('M3', 'M3 — Medium'),
    ('M4', 'M4 — Low'),
```

views.py (first lines)

```
from decimal import Decimal, InvalidOperation

from django.contrib import messages
from django.contrib.auth.decorators import login_required
from django.shortcuts import get_object_or_404, redirect, render
from django.views.decorators.http import require_POST

from projects.models import Project
from costing.models import CostingSheet
```

```

from .models import ProjectFinance, PaymentMilestone, CashOutflowRow

def _can_finance(user):
    """Finance workflow access — finance team and super admin only."""
    return bool(getattr(user, 'is_authenticated', False) and (
        user.is_super_admin_user or getattr(user, 'is_finance_team_user', False)))

def _parse_decimal(raw):
    raw = (raw or "").strip()

```

urls.py (first lines)

```

from django.urls import path

from . import views

app_name = 'finance'

urlpatterns = [
    path("", views.finance_home, name='home'),
    path('project/schedule/', views.project_schedule, name='schedule'),
    path('project/cash-outflow/', views.project_cash_outflow, name='cash_outflow'),
    path('margin-analysis/', views.margin_analysis_list, name='margin_analysis_list'),
    path('budgeting/', views.budgeting_list, name='budgeting_list'),
    path('budgeting/', views.sheet_budget, name='sheet_budget'),
    path('approve-margin/', views.approve_margin, name='approve_margin'),
]

```

admin.py (first lines)

```

from django.contrib import admin

from .models import ProjectFinance, PaymentMilestone

class PaymentMilestoneInline(admin.TabularInline):
    model = PaymentMilestone
    extra = 0

@admin.register(ProjectFinance)
class ProjectFinanceAdmin(admin.ModelAdmin):
    list_display = ['project', 'po_value', 'approved_margin', 'kickoff_date', 'updated_at']
    search_fields = ['project__project_name']
    inlines = [PaymentMilestoneInline]

```

tests.py (first lines)

```

from datetime import date
from decimal import Decimal

from django.test import TestCase
from django.urls import reverse

from accounts.models import User, Role
from projects.models import Project, ProjectStatus, Region
from costing.models import CostingSheet, CostingSection, CostingLineItem
from finance.models import ProjectFinance, PaymentMilestone

class FinanceScheduleTests(TestCase):
    def setUp(self):
        self.sa, _ = Role.objects.get_or_create(name=Role.SUPER_ADMIN)
        self.region = Region.objects.create(name='Arabia')
        self.user = User.objects.create_user('fin', password='pw', role=self.sa, region=self.region)
        self.client.force_login(self.user)
        self.won = ProjectStatus.objects.create(name='Won', category='won')
        self.project = Project.objects.create(

```

Quick troubleshooting hints:

- If a DB/schema bug: inspect C:\Users\Asadullah\OneDrive\Desktop\ERP\leap-erp\finance\models.py and migrations/ if present.

- If an endpoint or view logic bug: inspect C:\Users\Asadullah\OneDrive\Desktop\ERP\leap-erp\finance\views.py and C:\Users\Asadullah\OneDrive\Desktop\ERP\leap-erp\finance\urls.py
- If a form validation or input issue: inspect C:\Users\Asadullah\OneDrive\Desktop\ERP\leap-erp\finance\forms.py
- If an admin-only issue: inspect C:\Users\Asadullah\OneDrive\Desktop\ERP\leap-erp\finance\admin.py

App: fixtures

Path: C:\Users\Asadullah\OneDrive\Desktop\ERP\leap-erp\fixtures

****Primary purpose:**** (No module docstring found; inferred from filenames below)

Files:

- Ina_data.json

Key places to look when troubleshooting:

- models.py — database schema and model methods (data validation/business rules tied to persistence)
- views.py — request handling, business workflows, API endpoints and templates rendered
- forms.py — input validation and form-level logic
- urls.py — routing (which endpoint maps to which view)
- admin.py — admin UI customisations (if the bug appears in Django admin)

Quick troubleshooting hints:

- If a DB/schema bug: inspect C:\Users\Asadullah\OneDrive\Desktop\ERP\leap-erp\fixtures\models.py and migrations/ if present.
- If an endpoint or view logic bug: inspect C:\Users\Asadullah\OneDrive\Desktop\ERP\leap-erp\fixtures\views.py and C:\Users\Asadullah\OneDrive\Desktop\ERP\leap-erp\fixtures\urls.py
- If a form validation or input issue: inspect C:\Users\Asadullah\OneDrive\Desktop\ERP\leap-erp\fixtures\forms.py
- If an admin-only issue: inspect C:\Users\Asadullah\OneDrive\Desktop\ERP\leap-erp\fixtures\admin.py

App: hr

Path: C:\Users\Asadullah\OneDrive\Desktop\ERP\leap-erp\hr

****Primary purpose:**** (No module docstring found; inferred from filenames below)

Files:

- __init__.py
- admin.py
- apps.py
- attendance_exception_services.py
- attendance_matrix.py
- attendance_services.py
- forms.py
- leave_approval_services.py
- leave_services.py
- tests.py
- urls.py
- views.py
- work_calendar.py

Key places to look when troubleshooting:

- models.py — database schema and model methods (data validation/business rules tied to persistence)
- views.py — request handling, business workflows, API endpoints and templates rendered
- forms.py — input validation and form-level logic
- urls.py — routing (which endpoint maps to which view)
- admin.py — admin UI customisations (if the bug appears in Django admin)

views.py (first lines)

```
import json
from django.shortcuts import render, redirect, get_object_or_404
```

```

from django.contrib.auth.mixins import LoginRequiredMixin, UserPassesTestMixin
from django.contrib.auth.decorators import login_required
from django.views.decorators.http import require_POST
from django.contrib import messages
from django.views.generic import ListView, DetailView, CreateView, UpdateView, DeleteView, FormView
from django.urls import reverse_lazy, reverse
from django.db.models import Q, Count, Sum
from django.db.models import ProtectedError
from django.http import HttpResponse, JsonResponse
from django.utils import timezone
from django.db import transaction
from django.core.exceptions import PermissionDenied
from datetime import datetime, date, timedelta
import openpyxl
from openpyxl.styles import Font, Alignment, PatternFill, Border, Side

from .models import Employee, Asset, AssetAssignment, Vehicle, VehicleDocument, EmployeeDocument,
LeaveType, Holiday, LeaveEntitlement, LeaveRecord, AttendanceRecord, AttendanceSettings, WorkingDay,
WFHRecord, LeaveRequest, AttendanceException
from .forms import (

```

urls.py (first lines)

```

from django.urls import path
from . import views

app_name = 'hr'

urlpatterns = [
    path("", views.hr_dashboard, name='hr_dashboard'),
    path('my-profile/', views.my_profile, name='my_profile'),
    path('employees/', views.EmployeeListView.as_view(), name='employee_list'),
    # TEMPORARY bulk Office/Site back-fill — remove with the view + list controls.
    path('employees/bulk-work-location/', views.employee_bulk_work_location,
         name='employee_bulk_work_location'),
    path('create/', views.EmployeeCreateView.as_view(), name='employee_create'),
    path('import/', views.employee_import, name='employee_import'),
    path('export/', views.employee_export, name='employee_export'),
    path('/', views.EmployeeDetailView.as_view(), name='employee_detail'),
    path('/edit/', views.EmployeeUpdateView.as_view(), name='employee_update'),
    path('/delete/', views.EmployeeDeleteView.as_view(), name='employee_delete'),

    # Employee Documents
    path('/upload-document/', views.employee_document_upload, name='employee_doc_upload'),

```

forms.py (first lines)

```

from django import forms
from django.core.exceptions import ValidationError
from .models import Employee, Asset, AssetAssignment, Vehicle, EmployeeDocument, VehicleDocument,
LeaveType, Holiday, AttendanceSettings, WorkingDay, WFHRecord, AttendanceException

```

```

import re
from django.core.exceptions import ValidationError

NAME_RE = re.compile(
    r"^[A-Za-z\u00C0-\u017F]" # Latin + accented Latin
    r"\u0600-\u06FF" # Arabic
    r"\u0750-\u077F" # Arabic Supplement
    r"\u08A0-\u08FF" # Arabic Extended-A
    r"\uFB50-\uFDFF" # Arabic Presentation Forms-A
    r"\uFE70-\uFEFF" # Arabic Presentation Forms-B
    r"[\s\-\]\+\$"

```

admin.py (first lines)

```

from django.contrib import admin
from .models import Employee, Asset, LeaveType, LeaveEntitlement, LeaveRecord, Holiday,
AttendanceSettings, AttendanceRecord, WorkingDay, WFHRecord

@admin.register(Employee)
class EmployeeAdmin(admin.ModelAdmin):
    list_display = ['full_name', 'iqama_number', 'designation', 'nationality', 'deployment', 'contract_type', 'is_active']
    list_filter = ['contract_type', 'nationality', 'deployment', 'is_active']
    search_fields = ['full_name', 'iqama_number', 'work_email', 'mobile_number']

@admin.register(Asset)
class AssetAdmin(admin.ModelAdmin):
    list_display = ['asset_name', 'asset_type', 'serial_number', 'employee_name', 'condition', 'in_stock']
    list_filter = ['asset_type', 'condition', 'in_stock']
    search_fields = ['asset_name', 'serial_number', 'employee_name', 'invoice_number']

@admin.register(LeaveType)
class LeaveTypeAdmin(admin.ModelAdmin):

```

tests.py (first lines)

```

from datetime import date
from django.test import TestCase, skipUnlessDBFeature
from django.urls import reverse
from hr.work_calendar import count_working_days, is_working_day
from hr.forms import LeaveRequestForm
from hr.models import LeaveType
from hr.forms import LeaveRequestForm

from django.test import TestCase
from hr.forms import EmployeeForm, AssetForm

class EmployeeFormValidationTests(TestCase):
    """BUG-004: free-text fields must reject symbol-only garbage while
    still accepting legitimate Latin and Arabic names/titles."""
    def _base_valid_data(self, **overrides):
        data = {
            'full_name': 'John Smith',
            'nationality': 'British',

```

Quick troubleshooting hints:

- If a DB/schema bug: inspect C:\Users\Asadullah\OneDrive\Desktop\ERP\leap-erp\hr\models.py and migrations/ if present.
- If an endpoint or view logic bug: inspect C:\Users\Asadullah\OneDrive\Desktop\ERP\leap-erp\hr\views.py and C:\Users\Asadullah\OneDrive\Desktop\ERP\leap-erp\hr\urls.py
- If a form validation or input issue: inspect C:\Users\Asadullah\OneDrive\Desktop\ERP\leap-erp\hr\forms.py
- If an admin-only issue: inspect C:\Users\Asadullah\OneDrive\Desktop\ERP\leap-erp\hr\admin.py

App: kpis

Path: C:\Users\Asadullah\OneDrive\Desktop\ERP\leap-erp\kpis

****Primary purpose:**** (No module docstring found; inferred from filenames below)

Files:

- __init__.py
- activity.py
- activity_service.py
- admin.py
- apps.py
- models.py
- periods.py
- registry.py
- services.py

- tests.py
- urls.py
- views.py

Key places to look when troubleshooting:

- models.py — database schema and model methods (data validation/business rules tied to persistence)
- views.py — request handling, business workflows, API endpoints and templates rendered
- forms.py — input validation and form-level logic
- urls.py — routing (which endpoint maps to which view)
- admin.py — admin UI customisations (if the bug appears in Django admin)
- templates/ — HTML templates for UI (check for presentation/JS bugs)

models.py (first lines)

```
from django.db import models
from django.conf import settings
```

```
class KPIEntry(models.Model):
    """Per-period management input for one KPI.
```

Holds two optional numbers:

* ``target`` — the goal/threshold for the period (overrides the KPI's default target; for goal-based auto KPIs like revenue, this is the only place the goal lives).

* ``manual_value`` — the actual value for KPIs with no ERP source (the team types it). Ignored for auto KPIs, whose value is computed.

One row per (period, kpi_key). The KPI itself is defined in code (``kpis.registry``), so ``kpi_key`` is a loose CharField, not an FK.

```
"""
```

```
period = models.CharField(max_length=10, help_text="e.g. '2026-06', '2026-Q2', '2026'")
```

```
kpi_key = models.CharField(max_length=60)
```

```
target = models.DecimalField(max_digits=16, decimal_places=2, null=True, blank=True)
```

views.py (first lines)

```
from decimal import Decimal, InvalidOperation
```

```
from django.contrib import messages
```

```
from django.contrib.auth import get_user_model
```

```
from django.contrib.auth.decorators import login_required
```

```
from django.shortcuts import get_object_or_404, redirect, render
```

```
from django.urls import reverse
```

```
from accounts.permissions import require_capability
```

```
from .activity_service import (
    build_activity_overview, build_user_activity, activity_period_options,
)
```

```
from .models import KPIEntry
```

```
from .periods import current_period, period_options, period_bounds, label_for
```

```
from .registry import KPI_DEFINITIONS, DEPARTMENTS, KPI_BY_KEY
```

```
from .services import (
    build_dashboard, format_value, build_person_scorecard, attributable_users,
)
```

urls.py (first lines)

```
from django.urls import path
```

```
from . import views
```

```
app_name = 'kpis'
```

```
urlpatterns = [
```

```
    path("", views.dashboard, name='dashboard'),
```

```
    path('people/', views.people, name='people'),
```

```
    path('manage/', views.manage, name='manage'),
```

```

path('activity/', views.activity_overview, name='activity'),
path('activity//', views.activity_detail, name='activity_detail'),
]

```

admin.py (first lines)

```

from django.contrib import admin
from .models import KPIEntry

@admin.register(KPIEntry)
class KPIEntryAdmin(admin.ModelAdmin):
    list_display = ('period', 'kpi_key', 'target', 'manual_value', 'updated_by', 'updated_at')
    list_filter = ('period',)
    search_fields = ('kpi_key', 'period')

```

tests.py (first lines)

```

import datetime
from decimal import Decimal

from django.test import TestCase
from django.urls import reverse

from accounts.models import Role, RolePermission, User
from accounts.permissions import seed_default_permissions
from django.utils import timezone
from projects.models import Region, ProjectStatus, Project, ProjectHistory
from costing.models import CostingSheet, CostingSection, CostingLineItem
from procurement.models import PurchaseOrder, PurchaseOrderItem, POSummaryEntry

from .models import KPIEntry
from .periods import period_bounds, label_for
from .registry import (
    KPI_DEFINITIONS, KPI_BY_KEY, make_context, evaluate, achievement_pct,
    kpis_for_department, SALES, PROPOSAL, PROCUREMENT,
)
from .services import build_dashboard, build_person_scorecard, format_value

```

Quick troubleshooting hints:

- If a DB/schema bug: inspect C:\Users\Asadullah\OneDrive\Desktop\ERP\leap-erp\kpis\models.py and migrations/ if present.
- If an endpoint or view logic bug: inspect C:\Users\Asadullah\OneDrive\Desktop\ERP\leap-erp\kpis\views.py and C:\Users\Asadullah\OneDrive\Desktop\ERP\leap-erp\kpis\urls.py
- If a form validation or input issue: inspect C:\Users\Asadullah\OneDrive\Desktop\ERP\leap-erp\kpis\forms.py
- If an admin-only issue: inspect C:\Users\Asadullah\OneDrive\Desktop\ERP\leap-erp\kpis\admin.py

App: manpower

Path: C:\Users\Asadullah\OneDrive\Desktop\ERP\leap-erp\manpower

****Primary purpose:**** (No module docstring found; inferred from filenames below)

Files:

- __init__.py
- admin.py
- apps.py
- forms.py
- models.py
- urls.py
- views.py

Key places to look when troubleshooting:

- models.py — database schema and model methods (data validation/business rules tied to persistence)
- views.py — request handling, business workflows, API endpoints and templates rendered
- forms.py — input validation and form-level logic

- urls.py — routing (which endpoint maps to which view)
- admin.py — admin UI customisations (if the bug appears in Django admin)

models.py (first lines)

```
from datetime import date
from django.db import models
from django.conf import settings

class ManpowerSheet(models.Model):
    title = models.CharField(max_length=255, verbose_name='Title')
    project_reference = models.CharField(
        max_length=255, blank=True, verbose_name='Project Reference'
    )
    date = models.DateField(verbose_name='Date')
    notes = models.TextField(blank=True, verbose_name='Notes')
    created_by = models.ForeignKey(
        settings.AUTH_USER_MODEL,
        on_delete=models.SET_NULL,
        null=True, blank=True,
        related_name='manpower_sheets',
    )
    created_at = models.DateTimeField(auto_now_add=True)
```

views.py (first lines)

```
from django.shortcuts import render, redirect, get_object_or_404
from django.contrib.auth.mixins import LoginRequiredMixin, UserPassesTestMixin
from django.contrib.auth.decorators import login_required
from django.contrib import messages
from django.views.generic import ListView, DetailView, CreateView, UpdateView, DeleteView
from django.urls import reverse_lazy, reverse
from django.db.models import Q, Count, Sum, F
from django.http import HttpResponse
from django.utils.text import slugify
from datetime import datetime, date
import openpyxl
from openpyxl.styles import Font, Alignment, PatternFill, Border, Side

from .models import ManpowerSheet, ManpowerLineItem
from .forms import SheetForm, LineItemForm, EmployeeCreateForm, FilterForm, ImportForm
```

All cost field names used in DB-level aggregation

```
COST_FIELDS = [
    'gross_salary', 'iqama_cost', 'service_transfer_visa_fee',
    'gosi_cost', 'vacation_pay', 'exe_cost', 'eosb',
```

urls.py (first lines)

```
from django.urls import path
from . import views

app_name = 'manpower'

urlpatterns = [
    path("", views.SheetListView.as_view(), name='sheet_list'),
    path('create/', views.SheetCreateView.as_view(), name='sheet_create'),
    path('new-sheet/', views.NewSheetView.as_view(), name='sheet_create_new'),
    path('import/', views.sheet_import, name='sheet_import'),
    path('/', views.SheetDetailView.as_view(), name='sheet_detail'),
    path('/edit/', views.SheetUpdateView.as_view(), name='sheet_update'),
    path('/delete/', views.SheetDeleteView.as_view(), name='sheet_delete'),
    path('/export/', views.sheet_export, name='sheet_export'),
    path('/add-item/', views.LineItemCreateView.as_view(), name='lineitem_create'),
```

```

path('item//edit/', views.LineItemUpdateView.as_view(), name='lineitem_update'),
path('item//delete/', views.LineItemDeleteView.as_view(), name='lineitem_delete'),
]

```

forms.py (first lines)

```

from django import forms
from .models import ManpowerSheet, ManpowerLineItem

class SheetForm(forms.ModelForm):
    class Meta:
        model = ManpowerSheet
        fields = ['title', 'project_reference', 'date', 'notes']
        widgets = {
            'title': forms.TextInput(attrs={'class': 'form-control'}),
            'project_reference': forms.TextInput(attrs={
                'class': 'form-control',
                'placeholder': 'e.g. PRJ-2024-001',
            }),
            'date': forms.DateInput(attrs={
                'class': 'form-control',
                'type': 'date',
            }),
            'notes': forms.Textarea(attrs={
                'class': 'form-control',

```

admin.py (first lines)

```

from django.contrib import admin
from .models import ManpowerSheet, ManpowerLineItem

class ManpowerLineItemInline(admin.TabularInline):
    model = ManpowerLineItem
    extra = 1
    fields = [
        'order', 'employee_name', 'department', 'designation',
        'gross_salary', 'iqama_cost', 'gosi_cost', 'eosb',
    ]

@admin.register(ManpowerSheet)
class ManpowerSheetAdmin(admin.ModelAdmin):
    list_display = ['title', 'project_reference', 'date', 'created_by', 'created_at']
    list_filter = ['date']
    search_fields = ['title', 'project_reference']
    ordering = ['-date']
    inlines = [ManpowerLineItemInline]
    Relationships referenced (from scanned first lines):
    - FK -> settings.AUTH_USER_MODEL

```

Quick troubleshooting hints:

- If a DB/schema bug: inspect C:\Users\Asadullah\OneDrive\Desktop\ERP\leap-erp\manpower\models.py and migrations/ if present.
 - If an endpoint or view logic bug: inspect C:\Users\Asadullah\OneDrive\Desktop\ERP\leap-erp\manpower\views.py and C:\Users\Asadullah\OneDrive\Desktop\ERP\leap-erp\manpower\urls.py
 - If a form validation or input issue: inspect C:\Users\Asadullah\OneDrive\Desktop\ERP\leap-erp\manpower\forms.py
 - If an admin-only issue: inspect C:\Users\Asadullah\OneDrive\Desktop\ERP\leap-erp\manpower\admin.py
-

App: notifications

Path: C:\Users\Asadullah\OneDrive\Desktop\ERP\leap-erp\notifications

****Primary purpose:**** (No module docstring found; inferred from filenames below)

Files:

- __init__.py
- admin.py
- apps.py
- graph_email_backend.py
- models.py
- services.py
- signals.py
- urls.py
- views.py

Key places to look when troubleshooting:

- models.py — database schema and model methods (data validation/business rules tied to persistence)
- views.py — request handling, business workflows, API endpoints and templates rendered
- forms.py — input validation and form-level logic
- urls.py — routing (which endpoint maps to which view)
- admin.py — admin UI customisations (if the bug appears in Django admin)

models.py (first lines)

```
from django.db import models
from django.conf import settings
from django.contrib.contenttypes.fields import GenericForeignKey
from django.contrib.contenttypes.models import ContentType
```

```
class Notification(models.Model):
```

```
    LEVEL_CHOICES = [
        ('info', 'Info'),
        ('success', 'Success'),
        ('warning', 'Warning'),
        ('error', 'Error'),
    ]
```

```
    recipient = models.ForeignKey(
        settings.AUTH_USER_MODEL,
        on_delete=models.CASCADE,
        related_name='notifications',
    )
```

```
    actor = models.ForeignKey(
```

views.py (first lines)

```
from django.contrib.auth.decorators import login_required
from django.contrib.auth.mixins import LoginRequiredMixin
from django.http import JsonResponse
from django.views.decorators.http import require_POST
from django.views.generic import ListView
from .models import Notification
```

```
@login_required
```

```
def check_unread(request):
```

```
    """Return unread notification count for polling."""
```

```
    count = Notification.objects.filter(
        recipient=request.user, is_read=False
    ).count()
```

```
    return JsonResponse({'unread_count': count})
```

```
@login_required
```

```
def recent_notifications(request):
```

urls.py (first lines)

```

from django.urls import path
from . import views

app_name = 'notifications'

urlpatterns = [
    path("", views.NotificationListView.as_view(), name='list'),
    path('check/', views.check_unread, name='check'),
    path('recent/', views.recent_notifications, name='recent'),
    path('mark-read/', views.mark_read, name='mark_read'),
    path('mark-all-read/', views.mark_all_read, name='mark_all_read'),
]

```

admin.py (first lines)

```

from django.contrib import admin
from .models import Notification

@admin.register(Notification)
class NotificationAdmin(admin.ModelAdmin):
    list_display = ('recipient', 'actor', 'verb', 'level', 'is_read', 'email_sent', 'created_at')
    list_filter = ('level', 'is_read', 'email_sent', 'created_at')
    search_fields = ('verb', 'description', 'recipient__username')
    raw_id_fields = ('recipient', 'actor')
    readonly_fields = ('created_at',)

```

signals.py (first lines)

```

import logging

from django.db.models import Q
from django.db.models.signals import post_save
from django.dispatch import receiver
from django.urls import reverse

from accounts.models import User, Role
from reports.models import SalesCallResponse
from projects.models import ProjectHistory

from .services import notify_users

logger = logging.getLogger(__name__)

def _get_admins(region=None):
    """Admin recipients for a notification.

```

Super admins always receive (cross-region oversight). Regional admins Relationships referenced (from scanned first lines):
 - FK -> settings.AUTH_USER_MODEL

Quick troubleshooting hints:

- If a DB/schema bug: inspect C:\Users\Asadullah\OneDrive\Desktop\ERP\leap-erp\notifications\models.py and migrations/ if present.
 - If an endpoint or view logic bug: inspect C:\Users\Asadullah\OneDrive\Desktop\ERP\leap-erp\notifications\views.py and C:\Users\Asadullah\OneDrive\Desktop\ERP\leap-erp\notifications\urls.py
 - If a form validation or input issue: inspect C:\Users\Asadullah\OneDrive\Desktop\ERP\leap-erp\notifications\forms.py
 - If an admin-only issue: inspect C:\Users\Asadullah\OneDrive\Desktop\ERP\leap-erp\notifications\admin.py
-

App: procurement

Path: C:\Users\Asadullah\OneDrive\Desktop\ERP\leap-erp\procurement
 Primary purpose: (No module docstring found; inferred from filenames below)

Files:
 - __init__.py

- admin.py
- apps.py
- forms.py
- models.py
- quotation_extract.py
- tests.py
- urls.py
- views.py

Key places to look when troubleshooting:

- models.py — database schema and model methods (data validation/business rules tied to persistence)
- views.py — request handling, business workflows, API endpoints and templates rendered
- forms.py — input validation and form-level logic
- urls.py — routing (which endpoint maps to which view)
- admin.py — admin UI customisations (if the bug appears in Django admin)

models.py (first lines)

```
from django.db import models
from django.conf import settings
from decimal import Decimal

class PurchaseOrder(models.Model):
    """Purchase Order header with vendor info, project details, and terms."""

    STATUS_CHOICES = [
        ('draft', 'Draft'),
        ('issued', 'Issued'),
        ('acknowledged', 'Acknowledged'),
        ('completed', 'Completed'),
        ('cancelled', 'Cancelled'),
    ]

    COST_CENTER_CHOICES = [
        ('projects', 'Projects'),
        ('operations', 'Operations'),
        ('maintenance', 'Maintenance'),
```

views.py (first lines)

```
from django.views.generic import ListView, DetailView, CreateView, UpdateView, DeleteView
from django.contrib.auth.mixins import LoginRequiredMixin, UserPassesTestMixin
from django.shortcuts import get_object_or_404, redirect, render
from django.urls import reverse_lazy, reverse
from django.contrib import messages
from django.http import HttpResponse, JsonResponse
from django.views.decorators.http import require_POST
from django.db import transaction
from django.db.models import Q
from decimal import Decimal, InvalidOperation

from .models import (
    PurchaseOrder, PurchaseOrderItem,
    POSummaryEntry, QuotationImport,
    DeliveryNote, DeliveryNoteItem,
    InventoryReport, InventoryItem,
    FRCReport, FRCEntity, FRCInventory,
)
from .forms import (
    PurchaseOrderForm, PurchaseOrderItemForm, POItemFormSet, POFilterForm,
```

urls.py (first lines)

```
from django.urls import path
from . import views
```

```

app_name = 'procurement'
urlpatterns = [
    # Dashboard
    path("", views.procurement_dashboard, name='dashboard'),
    # Purchase Orders
    path('po/', views.POListView.as_view(), name='po_list'),
    path('po/create/', views.POCcreateView.as_view(), name='po_create'),
    # Import a supplier quotation PDF → AI extract → review → create PO.
    path('quotations/import/', views.quotation_import, name='quotation_import'),
    path('quotations//review/', views.quotation_review, name='quotation_review'),
    path('quotations//retry/', views.quotation_retry, name='quotation_retry'),
    path('budgets/', views.approved_budgets, name='approved_budgets'),
    path('po/from-bom/', views.po_create_from_bom, name='po_create_from_bom'),
    path('po/bom//tracker/', views.bom_procurement_tracker, name='bom_procurement_tracker'),
    path('po/import/', views.po_import_excel, name='po_import'),

```

forms.py (first lines)

```

from django import forms
from django.forms import inlineformset_factory
from .models import (
    PurchaseOrder, PurchaseOrderItem,
    DeliveryNote, DeliveryNoteItem,
    InventoryReport, InventoryItem,
    FRCReport, FRCEntry, FRCInventory,
)
from projects.models import Project

class PurchaseOrderForm(forms.ModelForm):
    class Meta:
        model = PurchaseOrder
        fields = [
            'po_date', 'po_number', 'cost_center', 'status',
            'vendor_name', 'vendor_contact_person', 'vendor_contact_email', 'vendor_contact_tel',
            'po_issued_by', 'issuer_email',
            'project', 'project_name', 'end_user', 'mr_item_number',
            'delivery_incoterms', 'delivery_location',

```

admin.py (first lines)

```

from django.contrib import admin
from .models import QuotationImport

@admin.register(QuotationImport)
class QuotationImportAdmin(admin.ModelAdmin):
    list_display = ['original_filename', 'status', 'model_used', 'purchase_order', 'created_by', 'created_at']
    list_filter = ['status']
    search_fields = ['original_filename']
    readonly_fields = ['extracted_data', 'model_used', 'error', 'created_at', 'updated_at']

```

tests.py (first lines)

```

import base64
import io
from datetime import date
from decimal import Decimal
from unittest import mock

from django.test import TestCase
from django.urls import reverse
from PIL import Image

from accounts.models import User, Role
from procurement.models import PurchaseOrder

```



```
def _png_data_url():
    buf = io.BytesIO()
    Image.new('RGBA', (8, 8), (0, 0, 0, 0)).save(buf, 'PNG')
    return 'data:image/png;base64,' + base64.b64encode(buf.getvalue()).decode()
```

Quick troubleshooting hints:

- If a DB/schema bug: inspect C:\Users\Asadullah\OneDrive\Desktop\ERP\leap-erp\procurement\models.py and migrations/ if present.
- If an endpoint or view logic bug: inspect C:\Users\Asadullah\OneDrive\Desktop\ERP\leap-erp\procurement\views.py and C:\Users\Asadullah\OneDrive\Desktop\ERP\leap-erp\procurement\urls.py
- If a form validation or input issue: inspect C:\Users\Asadullah\OneDrive\Desktop\ERP\leap-erp\procurement\forms.py
- If an admin-only issue: inspect C:\Users\Asadullah\OneDrive\Desktop\ERP\leap-erp\procurement\admin.py

App: projects

Path: C:\Users\Asadullah\OneDrive\Desktop\ERP\leap-erp\projects

****Primary purpose:**** (No module docstring found; inferred from filenames below)

Files:

- __init__.py
- admin.py
- apps.py
- forms.py
- models.py
- recovery.py
- tests.py
- urls.py
- views.py

Key places to look when troubleshooting:

- models.py — database schema and model methods (data validation/business rules tied to persistence)
- views.py — request handling, business workflows, API endpoints and templates rendered
- forms.py — input validation and form-level logic
- urls.py — routing (which endpoint maps to which view)
- admin.py — admin UI customisations (if the bug appears in Django admin)

models.py (first lines)

```
import re
```

```
from django.db import models
```

```
from django.conf import settings
```

Auto-generated LNA reference: "LNA - ", numbers

auto-incrementing from this floor. (Region code 'LNA'.)

```
LNA_REFERENCE_START = 2870
```

Canonical (new) name-bearing format: "LNA 2870 - Project Name" (optionally with

a trailing revision). Used to decide when it's safe to rebuild the reference

from the project name. Non-canonical refs (legacy codes, dash-joined imports)

only get their trailing revision swapped, never reformatted.

```
CANONICAL_LNA_RE = re.compile(r'^LNA \d+ - ')
```

```
LNA_REFERENCE_RE = CANONICAL_LNA_RE # back-compat alias
```

A trailing revision token in any style seen in the data:

" (R03)" · "-R03" · "- R03" · "_R03" · " R03"

```
_TRAILING_REV_RE = re.compile(
```

```
r'\s*(\s*R\d+\s*)[[-_]\s*R\d+|\sR\d+)\s*$', re.IGNORECASE)
```

The LNA number right after the 'LNA' prefix (optional separators / zero pad).

views.py (first lines)

```
from decimal import Decimal, InvalidOperation
```

```
from django.shortcuts import render, redirect, get_object_or_404
```

```
from django.http import Http404
```

```
from django.views.decorators.http import require_POST
```

```
from django.contrib.auth.decorators import login_required
```

```
from django.contrib.auth.mixins import LoginRequiredMixin, UserPassesTestMixin
```

```
from django.contrib import messages
```

```
from django.views.generic import ListView, DetailView, CreateView, UpdateView, DeleteView, View
```

```
from django.urls import reverse, reverse_lazy
```

```
from django.db.models import Q, Sum, Count
```

```
from django.core.paginator import Paginator
```

```
import openpyxl
```

```
from .models import Project, Region, ProjectStatus, ProjectHistory, Document, ProjectRevision
```

```
from .forms import ProjectForm, ProjectFilterForm, DocumentForm, DocumentFilterForm
```

```
from notifications.services import notify_users
```

```
from accounts.permissions import CapabilityRequiredMixin
```

urls.py (first lines)

```
from django.urls import path
```

```
from . import views
```

```
app_name = 'projects'
```

```
urlpatterns = [
```

```
# Project URLs
```

```
path("", views.ProjectListView.as_view(), name='list'),
```

```
path('recover/', views.project_recovery, name='recover'),
```

```
path('print/pdf/', views.pipeline_print_pdf, name='print_pdf'),
```

```
path('create/', views.ProjectCreateView.as_view(), name='create'),
```

```
path('/', views.ProjectDetailView.as_view(), name='detail'),
```

```
path('/edit/', views.ProjectUpdateView.as_view(), name='edit'),
```

```
path('/delete/', views.ProjectDeleteView.as_view(), name='delete'),
```

```
path('recycle-bin/', views.ProjectRecycleBinView.as_view(), name='recycle_bin'),
```

```
path('/restore/', views.project_restore, name='restore'),
```

```
path('import/', views.ProjectImportView.as_view(), name='import_projects'),
```

```
path('/add-document/', views.add_project_document, name='add_document'),
```

```
path('next-reference/', views.next_lna_reference_preview, name='next_reference_preview'),
```

forms.py (first lines)

```
from django import forms
from .models import (
    Project, Region, ProjectStatus, Document,
    build_lna_reference, next_lna_reference_number,
    parse_lna_reference, lna_reference_kind,
    split_trailing_revision, join_trailing_revision,
)

class ProjectForm(forms.ModelForm):
    """Form for creating/editing projects"""

    # Editable revision tag for LNA references, e.g. "R03" — appended to the
    # auto reference as "LNA ##### - (R03)". Not a model field; the
    # revision lives inside proposal_reference.
    lna_revision = forms.CharField(
        required=False, max_length=10,
        widget=forms.TextInput(attrs={'placeholder': 'e.g. R03', 'data-lna-revision': '1'}),
        help_text='Revision tag (optional), e.g. R03. Leave blank for none.')

```

admin.py (first lines)

```
from django.contrib import admin
from .models import Region, ProjectStatus, Project, ProjectHistory, Document

@admin.register(Region)
class RegionAdmin(admin.ModelAdmin):
    list_display = ['name', 'code', 'currency', 'is_active', 'created_at']
    list_filter = ['is_active', 'currency']
    search_fields = ['name', 'code']

@admin.register(ProjectStatus)
class ProjectStatusAdmin(admin.ModelAdmin):
    list_display = ['name', 'category', 'color', 'order', 'is_active']
    list_filter = ['category', 'is_active']
    search_fields = ['name']
    ordering = ['order', 'name']

@admin.register(Project)

```

tests.py (first lines)

```
from types import SimpleNamespace
from django.test import TestCase
from django.urls import reverse

from accounts.models import Role, User
from projects.models import Region, ProjectStatus, Project
from costing.models import CostingSheet, most_advanced_stage, pipeline_stage_badge
from django.test import TestCase, Client
from django.urls import reverse
from accounts.models import User, Role
from projects.models import Project, Region, ProjectStatus

class ProjectRegionFilterTests(TestCase):
    """BUG-001: users with no region assigned must not see other regions'
    projects, and should get a clear warning instead of a silent empty list."""

    def setUp(self):
        self.client = Client()

```

Quick troubleshooting hints:

- If a DB/schema bug: inspect C:\Users\Asadullah\OneDrive\Desktop\ERP\leap-erp\projects\models.py and migrations/ if present.

- If an endpoint or view logic bug: inspect C:\Users\Asadullah\OneDrive\Desktop\ERP\leap-erp\projects\views.py and C:\Users\Asadullah\OneDrive\Desktop\ERP\leap-erp\projects\urls.py
- If a form validation or input issue: inspect C:\Users\Asadullah\OneDrive\Desktop\ERP\leap-erp\projects\forms.py
- If an admin-only issue: inspect C:\Users\Asadullah\OneDrive\Desktop\ERP\leap-erp\projects\admin.py

App: proposals

Path: C:\Users\Asadullah\OneDrive\Desktop\ERP\leap-erp\proposals
Primary purpose: (No module docstring found; inferred from filenames below)

Files:

- __init__.py
- admin.py
- apps.py
- docx_export.py
- forms.py
- models.py
- pqd_export.py
- prequal_export.py
- prequal_views.py
- tests.py
- urls.py
- views.py

Key places to look when troubleshooting:

- models.py — database schema and model methods (data validation/business rules tied to persistence)
- views.py — request handling, business workflows, API endpoints and templates rendered
- forms.py — input validation and form-level logic
- urls.py — routing (which endpoint maps to which view)
- admin.py — admin UI customisations (if the bug appears in Django admin)

models.py (first lines)

```
from django.db import models
from django.conf import settings

class ProposalBoilerplate(models.Model):
    SECTION_CHOICES = [
        ('covering_letter', 'Covering Letter'),
        ('executive_summary', 'Executive Summary'),
        ('company_overview', 'Company Overview'),
        ('understanding_of_requirements', 'Understanding of Requirements'),
        ('proposed_technical_solution', 'Proposed Technical Solution'),
        ('delivery_implementation', 'Delivery & Implementation'),
        ('risk_management', 'Risk Management'),
        ('service_management', 'Service Management'),
        ('data_protection', 'Data Protection'),
        ('assumptions_constraints', 'Assumptions & Constraints'),
    ]
    name = models.CharField(max_length=255)
    section = models.CharField(max_length=40, choices=SECTION_CHOICES)
    content = models.TextField()
```

views.py (first lines)

```
from django.views.generic import ListView, DetailView, CreateView, UpdateView, DeleteView, View
from django.contrib.auth.mixins import LoginRequiredMixin, UserPassesTestMixin
from django.shortcuts import get_object_or_404, render, redirect
from django.urls import reverse_lazy, reverse
from django.contrib import messages
```

```

from django.http import JsonResponse
from django.views.decorators.http import require_POST
from django.contrib.auth.decorators import login_required
from django.db.models import Q, Max

from .models import (
    TechnicalProposal, ProposalBoilerplate,
    PrequalificationDocument, PQDAttachment,
    ProposalSection, SectionHeading,
)
from .forms import (
    ProposalMetadataForm, ProposalContentForm, EngineeringDocumentFormSet,
    ProposalSectionFormSet, ProposalFilterForm, ProposalBoilerplateForm,
    PQDMetadataForm, PQDFilterForm,
)

```

urls.py (first lines)

```

from django.urls import path
from django.contrib.auth.decorators import login_required
from . import views
from . import prequal_views

app_name = 'proposals'

urlpatterns = [
    # Dashboard (landing page for the Proposals section)
    path('dashboard/', views.proposals_dashboard, name='dashboard'),
    # Proposals
    path("", views.ProposalListView.as_view(), name='list'),
    path('create/', views.ProposalCreateView.as_view(), name='create'),
    path('/', views.ProposalDetailView.as_view(), name='detail'),
    path('/edit/', views.ProposalUpdateView.as_view(), name='edit'),
    path('/content/', views.ProposalEditContentView.as_view(), name='content'),
    path('/add-section/', views.add_proposal_section, name='add_section'),
    path('/delete/', views.ProposalDeleteView.as_view(), name='delete'),
    path('/export-docx/', login_required(views.proposal_export_docx), name='export_docx'),

```

forms.py (first lines)

```

from django import forms
from django.forms import inlineformset_factory
from .models import (
    TechnicalProposal, EngineeringDocument, ProposalBoilerplate,
    PrequalificationDocument, ProposalSection, SectionHeading,
)
from projects.models import Project

class ProposalSectionForm(forms.ModelForm):
    class Meta:
        model = ProposalSection
        fields = ['heading', 'content', 'order']
        widgets = {
            'heading': forms.TextInput(attrs={
                'class': 'form-control fw-bold', 'list': 'section-heading-options',
                'placeholder': 'Section heading'}),
            'content': forms.Textarea(attrs={'class': 'tinymce-editor'}),
            'order': forms.NumberInput(attrs={
                'class': 'form-control form-control-sm section-order', 'style': 'width:70px'}),

```

admin.py (first lines)

```

from django.contrib import admin
from .models import (
    TechnicalProposal, EngineeringDocument, ProposalBoilerplate,

```

```
SectionHeading, ProposalSection,
PrequalLibraryItem, PrequalSubmission,
)
```

```
@admin.register(PrequalLibraryItem)
class PrequalLibraryItemAdmin(admin.ModelAdmin):
    """The shared 25-PDF prequalification library — upload each heading's PDF."""
    list_display = ['order', 'heading', 'has_pdf', 'is_active', 'updated_at']
    list_editable = ['order', 'is_active']
    list_display_links = ['heading']
    list_filter = ['is_active']
    search_fields = ['heading']

    @admin.display(boolean=True, description='PDF')
    def has_pdf(self, obj):
        return bool(obj.pdf)
```

tests.py (first lines)

```
import io
import zipfile
from datetime import date

from django.test import TestCase
from django.urls import reverse

from proposals.models import (
    TechnicalProposal, ProposalSection, SectionHeading,
)
from proposals.docx_export import generate_proposal_docx
from accounts.models import User, Role
```

```
class ProposalDocxExportTests(TestCase):
    """The exported DOCX is built from the proposal's ProposalSection rows:
    one Heading1 + content block per section, in order. Rich text pasted from
    Word (tags with attributes like ) must render as formatted
    text, not dumped as literal HTML."""
```

Quick troubleshooting hints:

- If a DB/schema bug: inspect C:\Users\Asadullah\OneDrive\Desktop\ERP\leap-erp\proposals\models.py and migrations/ if present.
- If an endpoint or view logic bug: inspect C:\Users\Asadullah\OneDrive\Desktop\ERP\leap-erp\proposals\views.py and C:\Users\Asadullah\OneDrive\Desktop\ERP\leap-erp\proposals\urls.py
- If a form validation or input issue: inspect C:\Users\Asadullah\OneDrive\Desktop\ERP\leap-erp\proposals\forms.py
- If an admin-only issue: inspect C:\Users\Asadullah\OneDrive\Desktop\ERP\leap-erp\proposals\admin.py

App: reports

Path: C:\Users\Asadullah\OneDrive\Desktop\ERP\leap-erp\reports

****Primary purpose:**** (No module docstring found; inferred from filenames below)

Files:

- __init__.py
- admin.py
- apps.py
- forms.py
- models.py
- tests.py
- urls.py
- views.py

Key places to look when troubleshooting:

- models.py — database schema and model methods (data validation/business rules tied to persistence)
- views.py — request handling, business workflows, API endpoints and templates rendered
- forms.py — input validation and form-level logic
- urls.py — routing (which endpoint maps to which view)
- admin.py — admin UI customisations (if the bug appears in Django admin)

models.py (first lines)

```
from django.db import models
from django.conf import settings

class Vendor(models.Model):
    """Partner, Distributor & Vendor companies"""
    VENDOR_TYPE_CHOICES = [
        ('vendor', 'Vendor'),
        ('distributor', 'Distributor'),
        ('partner', 'Partner'),
        ('oem', 'OEM'),
    ]

    name = models.CharField(max_length=255)
    vendor_type = models.CharField(max_length=20, choices=VENDOR_TYPE_CHOICES, default='vendor')
    description = models.TextField(blank=True)
    website = models.URLField(max_length=500, blank=True)
    contact_person = models.CharField(max_length=255, blank=True)
    contact_email = models.EmailField(blank=True)
    contact_phone = models.CharField(max_length=50, blank=True)
```

views.py (first lines)

```
from django.shortcuts import render, redirect, get_object_or_404
from django.contrib.auth.decorators import login_required
from django.contrib import messages
from django.http import HttpResponse
from django.db.models import Sum, Count, Q
from django.core.paginator import Paginator
from django.views.generic import ListView, CreateView, UpdateView, DeleteView, DetailView
from django.contrib.auth.mixins import LoginRequiredMixin, UserPassesTestMixin
from django.urls import reverse_lazy
from openpyxl import Workbook, load_workbook
from openpyxl.styles import Font, PatternFill, Alignment, Border, Side
from io import BytesIO
from datetime import datetime, date, timedelta
from dateutil.parser import parse as parse_date

from projects.models import Project, Region, ProjectStatus
from accounts.decorators import manager_or_admin_required
from .models import Vendor, EPC, Exhibition, ProcurementPortal, Certification, SalesContact, SalesCallReport, SalesCallResponse
from .forms import SalesCallReportForm, SalesCallReportFilterForm
```

urls.py (first lines)

```
from django.urls import path
from . import views

app_name = 'reports'

urlpatterns = [
    path("", views.index, name='index'),
    path('export/', views.export_excel, name='export'),
    path('import/', views.import_excel, name='import'),
    path('summary/', views.summary_report, name='summary'),
    path('annual-report/', views.annual_report, name='annual_report'),
]

# Sales Call Reports
```

```

path('sales-calls/', views.SalesCallReportListView.as_view(), name='sales_call_list'),
path('sales-calls/add/', views.SalesCallReportCreateView.as_view(), name='sales_call_create'),
path('sales-calls/', views.SalesCallReportDetailView.as_view(), name='sales_call_detail'),
path('sales-calls/edit/', views.SalesCallReportUpdateView.as_view(), name='sales_call_update'),
path('sales-calls/delete/', views.SalesCallReportDeleteView.as_view(), name='sales_call_delete'),
path('sales-calls/export/', views.export_sales_call_reports, name='sales_call_export'),
path('sales-calls/print/pdf/', views.sales_call_print_pdf, name='sales_call_print_pdf'),

```

forms.py (first lines)

```

from django import forms
from .models import SalesCallReport

class SalesCallReportForm(forms.ModelForm):
    """Form for creating and editing Sales Call Reports"""

    call_date = forms.DateField(
        widget=forms.DateInput(attrs={'type': 'date', 'class': 'form-control'}),
        help_text="Date of the call/meeting"
    )

    next_action_date = forms.DateField(
        required=False,
        widget=forms.DateInput(attrs={'type': 'date', 'class': 'form-control'}),
        help_text="Scheduled date for next action"
    )

    system_categories = forms.MultipleChoiceField(
        choices=SalesCallReport.SYSTEM_CATEGORY_CHOICES,

```

admin.py (first lines)

```

from django.contrib import admin
from .models import Vendor, EPC, Exhibition, ProcurementPortal, Certification, SalesContact, SalesCallReport,
SalesCallResponse

@admin.register(Vendor)
class VendorAdmin(admin.ModelAdmin):
    list_display = ['name', 'vendor_type', 'contact_person', 'is_active', 'created_at']
    list_filter = ['vendor_type', 'is_active']
    search_fields = ['name', 'contact_person', 'products_services']
    ordering = ['name']

@admin.register(EPC)
class EPCAdmin(admin.ModelAdmin):
    list_display = ['name', 'region', 'contact_person', 'is_active', 'created_at']
    list_filter = ['is_active', 'region']
    search_fields = ['name', 'contact_person', 'specialization']
    ordering = ['name']

```

tests.py (first lines)

```

from django.test import TestCase

```

Create your tests here.

Quick troubleshooting hints:

- If a DB/schema bug: inspect C:\Users\Asadullah\OneDrive\Desktop\ERP\leap-erp\reports\models.py and migrations/ if present.
- If an endpoint or view logic bug: inspect C:\Users\Asadullah\OneDrive\Desktop\ERP\leap-erp\reports\views.py and C:\Users\Asadullah\OneDrive\Desktop\ERP\leap-erp\reports\urls.py
- If a form validation or input issue: inspect C:\Users\Asadullah\OneDrive\Desktop\ERP\leap-erp\reports\forms.py
- If an admin-only issue: inspect C:\Users\Asadullah\OneDrive\Desktop\ERP\leap-erp\reports\admin.py

Cross-app Relationships (inferred)

devtracking

- FK -> settings.AUTH_USER_MODEL

drafts

- FK -> ContentType
- FK -> content_type
- FK -> settings.AUTH_USER_MODEL

manpower

- FK -> settings.AUTH_USER_MODEL

notifications

- FK -> settings.AUTH_USER_MODEL

High-level Architecture and How Apps Integrate

The Projects app is the central model: projects.Project is referenced widely (procurement, finance, delivery, inventory, proposals).

Procurement manages PurchaseOrder and PurchaseOrderItem; these reference projects and feed into delivery notes, inventory reports, and finance cash outflow rows.

Finance contains ProjectFinance, PaymentMilestone and CashOutflowRow — it depends on costing and projects. Costing produces CostingSheet / line items used by finance and procurement.

Contacts and Reports provide CRM and sales call data used to seed Projects and Proposals. Accounts stores User and Role definitions and permission logic used across the app.

For a bug in HR:

- Start at hr/views.py for request-handling and API endpoints.
- Check hr/forms.py for validation rules that may reject input.
- Check hr/models.py for schema issues and business logic (signals, manager methods).
- Check templates/hr/ for UI problems and static/js for client-side bugs.

For database schema mismatches: examine migrations in each app (migrations/ folder) and db.sqlite3 or your configured DB. Run python manage.py makemigrations && migrate in a dev environment.

Recommended next actions

- I can expand each app section with deeper file-by-file explanations, extract model fields and relationships, and produce an ER diagram.
- Or generate this guide as a PDF now. (Will produce developer_guide.pdf)